Anchor-Based Item Extraction from SEC 10-K Filings

Eric Bi

March 17, 2026

Version	*Date*	*Comments*
1.0	March 17, 2026	Initial release
1.1	March 18, 2026	Add document-level retrieval rate; DP scoring and shared-anchor improvements

Abstract

We present a rule-based pipeline for extracting standardized item sections from SEC 10-K filings. The method exploits the anchor-link structure of the HTML Table of Contents to identify section boundaries, classifies anchors via a three-tier regex hierarchy, and resolves ordering conflicts through a dynamic programming formulation of the weighted longest increasing subsequence problem. On a corpus of 392 annotated filings across three evaluation sets, the pipeline achieves a mean character-level F_1 score of **97.5%** and a document-level retrieval rate of **70.9%** (278/392), where a document is "retrieved" only when every ground truth item is present, no false positives exist, and all item boundaries exceed $F_1 \geq 0.9$. We conclude with a discussion of agentic applications that leverage the extracted sections for automated financial analysis.

Contents

1 Introduction **3**

2 Method **4**

2.1 Stage 1: Document Isolation 4

2.2 Stage 2: TOC Anchor Collection 4

2.3 Stage 3: Anchor Element Localization 4

2.4 Stage 4: Anchor Classification 4

2.4.1 Sequence-Constrained Selection via Dynamic Programming 5

2.5 Stage 5: HTML Slicing 5

2.5.1 Special Cases 5

3 Evaluation Protocol **6**

3.1 Dataset 6

3.2 Metric: Character-Level F_1 6

3.3 Metric: Document-Level Retrieval Rate 7

4 Results **7**
4.1 Overall Performance 7
4.2 Per-Item Performance 7
4.3 Error Analysis 8

5 Future Directions: Agentic Applications **9**
5.1 Multi-Agent Risk Assessment System 9
5.2 Autonomous Due Diligence Agent 9
5.3 Regulatory Compliance Monitoring Agent 10
5.4 Conversational Financial Analyst Agent 10
5.5 Portfolio Surveillance Agent Swarm 11

6 Conclusion **11**

1 Introduction

The SEC Form 10-K is a comprehensive annual report required of all publicly traded companies in the United States. Each 10-K follows a prescribed structure of approximately 20 item sections (see Table 1), spanning topics from business operations and risk factors to executive compensation and financial statements **zhang2023form10k**.

Automated extraction of individual item sections is a prerequisite for downstream tasks such as financial text classification, risk analysis, and regulatory compliance monitoring. However, 10-K filings exhibit substantial formatting heterogeneity: companies use different HTML structures, anchor naming conventions, and section delineation strategies, making robust extraction non-trivial.

This report describes a deterministic, rule-based extraction pipeline that achieves 97.5% mean character-level F_1 and 70.9% document-level retrieval rate across 392 evaluated filings without reliance on trained models. We detail the method (Section 2), the evaluation protocol (Section 3), experimental results (Section 4), and future directions toward agentic applications (Section 5).

Table 1: Standard 10-K item sections.

Item	Description
Item 1	Business overview
Item 1A	Risk factors
Item 1B	Unresolved staff comments
Item 2	Properties
Item 3	Legal proceedings
Item 4	Mine safety disclosures
Item 5	Market for registrant's common equity
Item 6	Selected financial data / Reserved
Item 7	Management's discussion and analysis
Item 7A	Quantitative and qualitative disclosures about market risk
Item 8	Financial statements and supplementary data
Item 9	Changes in and disagreements with accountants
Item 9A	Controls and procedures
Item 9B	Other information
Item 9C	Disclosure regarding foreign jurisdictions
Items 10–14	Directors, executive compensation, security ownership, certain relationships, principal accountant fees
Item 15	Exhibits and financial statement schedules
Item 16	Form 10-K summary
Signatures	Signatory attestation

2 Method

The pipeline consists of five sequential stages. The input is a raw SEC EDGAR submission file (`.txt`); the output is a dictionary mapping each identified item to its corresponding HTML slice.

2.1 Stage 1: Document Isolation

SEC submission files may contain multiple embedded documents (the 10-K proper, exhibits, XBRL data, graphics). We extract the 10-K document by searching for the `<DOCUMENT>` block whose `<TYPE>` field matches `10-K` or `10-K/A`, then isolate the HTML content within its `<TEXT>` element.

2.2 Stage 2: TOC Anchor Collection

We scan the HTML for all internal hyperlinks of the form `<a href="#ID">`. The set of referenced anchor IDs constitutes the candidate pool. This filtering step is essential: a typical 10-K contains hundreds of `id` attributes for footnotes, cross-references, and formatting; restricting to TOC-referenced anchors reduces the search space by an order of magnitude.

2.3 Stage 3: Anchor Element Localization

For each ID in the candidate pool, we locate the corresponding HTML element by matching `id` or `name` attributes across all tag types (`<a>`, `<div>`, `<p>`, `<span>`, `<td>`, etc.). When duplicate IDs exist, we retain the *last* occurrence, as the first typically resides in the TOC header while the last marks the actual section body.

2.4 Stage 4: Anchor Classification

Each anchor is classified to a 10-K item via a three-tier system applied in decreasing order of confidence:

Tier 0: Anchor ID matching. Regex patterns applied to the anchor's `id` string itself. Many filings use self-describing IDs such as `ITEM1ARISKFACTORS` or `item_7a`. This tier has the highest precision.

Tier 1: Explicit item patterns. Regex patterns matching "Item X" text in the forward lookahead window (up to 2000 characters or the next anchor, whichever is smaller). Patterns are ordered by specificity: longer sub-item patterns (e.g., "Item 9A") precede shorter base patterns (e.g., "Item 9") to avoid false matches.

Tier 2: Descriptive title matching. For filings that omit explicit item numbering, we match characteristic phrases such as "Risk Factors" (Item 1A), "Management's Discussion and Analysis" (Item 7), or "Executive Compensation" (Item 11). This tier has the lowest confidence and is applied only when Tiers 0 and 1 yield no match.

A heuristic filter rejects anchors that exhibit characteristics of TOC entries rather than section headings (e.g., anchor IDs containing "toc", or forward text containing ≥ 5 distinct item mentions).

2.4.1 Sequence-Constrained Selection via Dynamic Programming

Items in a 10-K must appear in a prescribed sequential order. Let $\mathcal{C} = \{(o_i, s_i, c_i)\}_{i=1}^{N}$ denote the set of candidate anchors, where o_i is the character offset, $s_i \in \{1, 2, \ldots, 24\}$ is the sequential item index, and c_i is the classification confidence. We seek a subset $\mathcal{A} \subseteq \mathcal{C}$ such that:

1. offsets are strictly increasing: $o_i < o_j$ for all $i < j$ in the selected order;
2. item indices are strictly increasing: $s_i < s_j$;
3. the total score $\sum_{i \in \mathcal{A}} w(c_i, o_i)$ is maximized, where $w(c, o) = c \cdot 10^6 + (o_{\max} - o)$.

This is equivalent to the **weighted longest increasing subsequence** (WLIS) problem over the pair (s_i, o_i), which we solve via $O(N^2)$ dynamic programming. The weight function w prioritizes classification confidence while preferring *earlier* offsets among same-confidence candidates as a tiebreaker. This design choice reflects the observation that later occurrences of a given anchor classification are more likely to be cross-references than true section headings, since the last physical occurrence of each anchor ID (the body copy) is already selected in Stage 3.

2.5 Stage 5: HTML Slicing

Given the ordered anchor sequence $\{(o_k, \text{item}_k)\}_{k=1}^{K}$, we extract the HTML substring $h[o_k : o_{k+1}]$ for each item k. Boundary adjustments handle two edge cases:

- **Wrapper `<div>` elements**: when an anchor is wrapped in `<div><a id="..."></a></div>`, the previous item's slice extends through the closing `</div>`, and the current item's slice begins at the `<a>` tag.
- **Terminal item**: the final item extends to `</body>` or the end of the document.

2.5.1 Special Cases

Three categories of items receive special handling:

- **Signatures**: When detected via a TOC-referenced anchor, the full slice content is output. When detected only via a fallback bold-text scan (no TOC link), the entry is suppressed to avoid false positives. Ground truth is empty for 92% of filings; the evaluation metric assigns $F_1 = 1.0$ for empty ground truth regardless of prediction, so outputting content is safe.
- **Item 16**: Subject to placeholder detection. When signatures are suppressed and the item contains only placeholder text ("None", "Not applicable") with a large slice (>10K characters, indicating absorption of subsequent exhibits), the value is set to empty.
- **Shared-anchor item groups**: When multiple TOC entries point to a single anchor (common for Part III Items 10–14 incorporated by reference, and occasionally for Items 9/9A/9B or Items 1/1A), the content is assigned to the last item in the group and empty placeholder keys are emitted for the remaining items.

3 Evaluation Protocol

3.1 Dataset

The evaluation corpus comprises three sets of 167 SEC EDGAR submission files each. After excluding filings with empty ground truth annotations, the evaluated subset contains 133, 132, and 127 filings for Sets 1, 2, and 3, respectively (392 filings total).

Ground truth annotations consist of JSON dictionaries mapping `accession#item_name` keys to verbatim HTML substrings extracted from the source filing.

3.2 Metric: Character-Level F_1

We employ character-level F_1 computed over bag-of-character representations. For a predicted string $\hat{y}$ and ground truth string y, let $\hat{t} = \text{strip}(\hat{y})$ and $t = \text{strip}(y)$ denote the plain-text representations obtained by removing all HTML tags and decoding all HTML entities via `html.unescape()`. Define character frequency vectors $\boldsymbol{p}, \boldsymbol{g} \in \mathbb{N}^{|\Sigma|}$ over the character alphabet Σ. Then:

$$P = \frac{\sum_{c \in \Sigma} \min(p_c, g_c)}{\sum_{c \in \Sigma} p_c}, \quad R = \frac{\sum_{c \in \Sigma} \min(p_c, g_c)}{\sum_{c \in \Sigma} g_c}, \quad F_1 = \frac{2PR}{P + R} \tag{1}$$

The overall score is the macro-average: for each filing, we compute the mean F_1 over all items present in the ground truth, then average across all filings.

Entity normalization. The `strip()` function applies `html.unescape()` to decode all HTML character references (e.g., ` ` → U+00A0) prior to character counting. This ensures that equivalent representations of the same character are not penalized as mismatches.

3.3 Metric: Document-Level Retrieval Rate

Following Zhang et al. **zhang2023form10k**, we define a stricter document-level metric. A document is considered "retrieved" only when *all three* of the following conditions hold simultaneously:

1. **No missing items**: every ground truth item key is present in the prediction;
2. **No false positives**: no predicted keys exist that are absent from the ground truth;
3. **Correct boundaries**: every item's character-level $F_1 \geq 0.9$.

The retrieval rate is the fraction of evaluated documents that satisfy all three conditions. This metric is substantially more demanding than mean F_1: a single missing item, spurious key, or boundary error on any item causes the entire document to fail.

4 Results

4.1 Overall Performance

Table 2: Overall performance metrics.

Dataset	Files	F_1	Extraction Rate	Doc Retrieval
Set 1	133	98.0%	98.8%	74.4%
Set 2	132	97.5%	98.2%	68.2%
Set 3	127	97.1%	97.7%	70.1%
Mean	392	**97.5%**	98.2%	**70.9%**

The pipeline achieves a mean F_1 of 97.5% across all three evaluation sets, with extraction rates (fraction of ground truth items for which a non-empty prediction exists) exceeding 97% in all cases. The document-level retrieval rate—requiring all items present, no false positives, and all boundaries at $F_1 \geq 0.9$—is 70.9% (278/392). The gap between high per-item F_1 and moderate retrieval rate reflects the metric's strict conjunctive nature: a single item error in any of the $\sim$20 items per filing causes the entire document to fail.

4.2 Per-Item Performance

Table 3 presents per-item F_1 scores for Set 1. Performance exceeds 95% for 14 of the 17 item categories.

Performance exceeds 95% for 16 of the 17 item categories. Only **Signatures** (92.4%) falls below this threshold. The pipeline outputs the full anchor-to-anchor slice for signatures when a

Table 3: Per-item character-level F_1 on Set 1 (133 filings).

Item	F_1	n	Item	F_1	n
item1	99.3%	131	item9	99.3%	129
item1a	99.2%	132	item9a	100.0%	131
item1b	99.2%	126	item9b	97.8%	126
item2	99.2%	128	item10	99.2%	131
item3	99.6%	132	item11	99.4%	130
item4	98.4%	126	item12	99.8%	131
item5	98.6%	129	item13	98.5%	130
item6	97.1%	131	item14	98.2%	130
item7	97.7%	130	item15	100.0%	133
item7a	96.4%	129	item16	95.8%	106
item8	98.6%	130	signatures	92.4%	105

TOC-referenced anchor exists; however, 8 filings in Set 1 have ground truth spanning 12M–80M characters (the full remainder of the submission including all exhibits), which is unmatchable by section-boundary methods. Items previously below 95%—Item 16 and Item 6—improved through refined shared-anchor handling and the prefer-earlier DP scoring heuristic.

4.3 Error Analysis

Of the 114 documents that fail the retrieval criterion, the failure modes decompose as follows:

- **Boundary errors only** (37 docs): All items are present with no false positives, but at least one item has $F_1 < 0.9$. The most affected items are signatures (15 docs, where ground truth extends into exhibit content beyond the 10-K document boundary), item 16 (9 docs, inconsistent ground truth conventions), and item 9B (5 docs, trailing Part III header absorption).
- **Missing items only** (21 docs): Dominated by signatures (13 docs), where the filing lacks a TOC entry for signatures and the fallback detector is suppressed to avoid false positives.
- **False positives only** (10 docs): Extra predicted items not in ground truth, primarily item 16 (4 docs) in filings where the ground truth omits this optional section.
- **Mixed failures** (46 docs): Combinations of the above, often involving filings with non-standard HTML structure.

The dominant irreducible error source is the *exhibit content boundary*: ground truth for signatures and item 16 sometimes extends into exhibit documents (`EX-31`, `EX-32`, etc.) that reside in separate `<DOCUMENT>` blocks within the SEC submission. Since the extraction pipeline operates on the

primary 10-K document only, this content is structurally inaccessible. Approximately 30 of the 114 failures are attributable to this limitation.

5 Future Directions: Agentic Applications

The extraction pipeline produces structured, per-item representations of 10-K filings at 97.5% fidelity with 70.9% document-level retrieval. This structured output serves as a foundation for autonomous agent systems that perform financial analysis tasks requiring targeted access to specific filing sections **zhang2023form10k**. We outline several agentic architectures that build upon the pipeline.

5.1 Multi-Agent Risk Assessment System

A system of cooperating agents could perform automated risk analysis by orchestrating specialized sub-agents:

Risk Extraction Agent. Consumes the Item 1A (Risk Factors) slice and applies named entity recognition, topic modeling, and sentiment analysis to produce a structured risk taxonomy per filing.

Temporal Comparison Agent. Given risk taxonomies from consecutive fiscal years, identifies newly emerged risks, resolved risks, and risks with changed severity language. Outputs a year-over-year risk delta report.

Cross-Company Aggregation Agent. Ingests risk taxonomies across an industry sector (e.g., all S&P 500 technology companies) and identifies systemic risks that appear across multiple filings, ranking by prevalence and severity.

Orchestrator Agent. Coordinates the above sub-agents, manages the extraction pipeline invocations, and synthesizes a final risk briefing document for a human analyst.

5.2 Autonomous Due Diligence Agent

An agent tasked with pre-acquisition due diligence could consume multiple item sections in a structured workflow:

1. Extract Item 1 (Business), Item 3 (Legal Proceedings), Item 7 (MD&A), and Item 8 (Financial Statements) for the target company's last five fiscal years.
2. Identify material litigation trends from Item 3 across years.

3. Extract revenue segments, margins, and management outlook from Item 7 and correlate with Item 8 quantitative data.
4. Produce a structured due diligence memo with flagged anomalies, each linked to the source filing and item section.

The pipeline's per-item extraction eliminates the need for the agent to process entire filings (often 10M+ characters), reducing context window consumption by 95% while preserving section-level provenance.

5.3 Regulatory Compliance Monitoring Agent

A continuously running agent could monitor SEC EDGAR for new filings and perform automated compliance checks:

Filing Ingestion. Polls EDGAR for new 10-K submissions, runs the extraction pipeline, and stores per-item content in a vector database with metadata (company, fiscal year, item, CIK).

Policy Enforcement. Compares extracted Item 9A (Controls and Procedures) and Item 9C (Foreign Jurisdiction Disclosures) against a configurable rule set encoding regulatory requirements. Flags filings that omit required disclosures or contain boilerplate language below a specificity threshold.

Alert Generation. Routes flagged filings to compliance officers with direct links to the relevant extracted section, reducing manual review time from hours (full filing) to minutes (targeted section).

5.4 Conversational Financial Analyst Agent

An LLM-based conversational agent could use the extraction pipeline as a tool within a retrieval-augmented generation (RAG) architecture:

1. The user asks: "What are Apple's main risk factors in their 2024 10-K, and how do they compare to 2023?"
2. The agent invokes the pipeline to extract Item 1A from both filings.
3. The extracted sections (typically 10–50 pages each) are chunked, embedded, and indexed in a session-scoped vector store.
4. The agent retrieves relevant chunks and synthesizes a comparative analysis, citing specific passages with page-level provenance.

By extracting the relevant section before retrieval, the agent avoids the noise and irrelevance that arises from embedding and searching entire filings. This *extract-then-retrieve* pattern improves both precision and citation accuracy compared to naive full-document RAG.

5.5 Portfolio Surveillance Agent Swarm

A swarm of lightweight agents could monitor a portfolio of holdings:

Watcher Agents (one per holding): Each agent tracks its assigned company's EDGAR filings. When a new 10-K is published, it extracts all items and computes a diff against the prior year's extracted sections.

Anomaly Detection Agent: Receives diffs from all Watcher Agents. Applies statistical and semantic anomaly detection to identify unusual changes: significant new risk factors, large swings in reported segment revenue, changes in auditor, new legal proceedings, or removal of previously disclosed items.

Portfolio Manager Agent: Aggregates anomaly reports, cross-references with market data and news, and produces a daily portfolio surveillance briefing ranked by materiality.

This architecture scales linearly with portfolio size and requires no human intervention for routine monitoring, reserving analyst attention for flagged anomalies.

6 Conclusion

We presented a deterministic, rule-based pipeline for extracting standardized item sections from SEC 10-K filings. The method exploits the anchor-link structure of HTML Tables of Contents, classifies anchors via a three-tier regex hierarchy, and resolves ordering constraints via dynamic programming. On 392 evaluated filings, the pipeline achieves a mean character-level F_1 of 97.5% and a document-level retrieval rate of 70.9%, with all three evaluation sets exceeding 96% mean F_1.

The high-fidelity, per-item structured output produced by this pipeline serves as an enabling primitive for agentic financial analysis systems. Multi-agent architectures for risk assessment, due diligence, compliance monitoring, conversational analysis, and portfolio surveillance can leverage targeted section extraction to reduce context window consumption, improve retrieval precision, and maintain provenance to source filings.